

Project Title

Project Team

Student 1 19I-1234

Student 2 19I-1234

Student 3 19I-1234

Session 2018-2022

Supervised by

Mr./ Ms./ Dr. Supervisor Name

Co-Supervised by

Mr./ Ms./ Dr. Supervisor Name



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2022

Contents

1	Introduction [AS PER SCOPE DOCUMENT]	1
1.1	Existing Solutions	1
1.2	Problem Statement	2
1.3	Scope	4
1.4	Modules	5
1.4.1	ForeSyte Web Portal	5
1.4.2	Backend Service	6
1.5	Work Division	6
2	Project Requirements [AS PER FYP1 MID REPORT]	9
2.1	Use-case/Event Response Table/Storyboarding	9
2.2	Functional Requirements	10
2.2.1	Module 1	10
2.2.2	Module 2	10
2.3	Non-Functional Requirements	10
2.3.1	Usability	10
2.3.2	Performance	11
3	System Overview [AS PER FYP1 MID REPORT]	13
3.1	Architectural Design	13
3.2	Data Design	14
3.3	Domain Model	14
3.4	Design Models [UPTO THE CURRENT ITERATION ONLY]	15
4	Implementation and Testing	17
4.1	Current Progress	17
4.1.1	Front-End	17
4.1.2	Back-End	18
4.1.3	Next Steps	20
4.2	Algorithm Design	21
4.2.1	Admin Dashboard Aggregation	21
4.2.2	Report Generation Workflow (Planned Design)	22
4.2.3	Investigator Violation Review Flow	23

4.3	External APIs and Libraries	23
4.4	Testing Details	23
4.4.1	Unit Testing	24
References		34
A Appendices		35
A.1	Appendix A	35
A.1.1	Use Case Diagram example (Online Shopping System)	35
A.1.2	Detail Use Case Example	36
A.1.3	Event-Response Table for a Highway Intersection	37
A.1.4	Story Board Example For Android App	38
A.2	Appendix B	39
A.2.1	Domain Model Example For Online Shopping	39
A.3	Appendix C	40
A.3.1	Box And Line Example For Online Shopping	40
A.3.2	Architecture Pattern Example For Online Shopping	41
A.4	Appendix D	42
A.4.1	Activity Diagram	42
A.4.2	Class Diagram	43
A.4.3	Sequence Diagram	44
A.4.4	State Transition Diagram	44
A.4.5	Data Flow Diagram	45

List of Figures

1.1	Estimated reduction in cheating rates using ForeSyte AI-based surveillance system	3
1.2	Frequency of students who reported cheating during written examinations [3]	3
1.3	Scope workflow of the ForeSyte system	4
A.1	Use Case Diagram for the Online Shopping System	35
A.2	Detail Use Case Example	36
A.3	Example of Event Response Table	37
A.4	Example of Story Board	38
A.5	Domain Model Example For Online Shopping Application	39
A.6	Box and Line Diagram For Online Shopping Application	40
A.7	Architecture Pattern For Online Shopping Application	41
A.8	Activity Diagram For Online Shopping Application	42
A.9	Class Diagram For Online Shopping Application	43
A.10	Sequence Diagram For Online Shopping Application	44
A.11	State Transition Diagram For Online Shopping Application	44

List of Tables

1.1	Comparison of Existing Solutions	2
1.2	Work Division among Team Members	7
4.1	Unit Test Cases for Reports API	25
4.2	Unit Test Cases for Dashboard APIs	27
4.3	Unit Test Cases for User Management API	29
4.4	Unit Test Cases for Seating Plan Upload	31
4.5	Unit Test Cases for Monitoring and Camera Integration	33

Chapter 1

Introduction [AS PER SCOPE DOCUMENT]

The "Introduction" section sets the foundation for the entire report and should be carefully crafted to give the reader a clear understanding of the project's scope, objectives, and context. Here's what you can include:

Background:

Begin by providing background information on the broader topic or field your development project is related to. Highlight any existing problems, gaps, or opportunities that led to the motivation for your project. Include relevant references to previous research, development projects, or technologies. For example, you might want to cite studies or prior systems that failed to address the issues you're focusing on.

1.1 Existing Solutions

For the Existing Solutions section, students should research and describe various solutions, methods, or approaches that have been previously developed to solve the problem they are addressing. This section should include an overview of the different techniques, technologies, or models that are currently in use or have been applied in the past. Students should critically assess the strengths and weaknesses of each solution, highlighting any gaps or limitations that their project aims to address. Additionally, this section should show an understanding of the state-of-the-art in the field and provide context for why the proposed solution is necessary or an improvement over existing ones.

Table 1.1: Comparison of Existing Solutions

System Name	System Overview	System Limitations
System 1	Brief description of what System 1 does and its primary features.	Highlight the shortcomings or issues with System 1.
System 2	Brief description of what System 2 does and its primary features.	Highlight the shortcomings or issues with System 2.
System 3	Brief description of what System 3 does and its primary features.	Highlight the shortcomings or issues with System 3.

1.2 Problem Statement

Academic dishonesty has become a significant concern in educational institutions worldwide. Studies indicate that approximately 61% of university students have admitted to cheating during written examinations, while nearly 68% confess to cheating at least once during their academic career [3]. Traditional manual invigilation methods are often insufficient, as invigilators may unintentionally overlook suspicious activities, and reviewing long hours of CCTV/camera footage after exams is both time-consuming and ineffective.

To address these challenges, **ForeSyte** provides an AI-driven solution for automated examination monitoring. The system processes both real-time CCTV/camera streams and uploaded video footage, detecting suspicious activities such as phone usage, hand waving, bending over the desk, looking around, and standing up using advanced computer vision techniques.

The system focuses on seat-specific mapping, ensuring that every detected activity is linked to the correct student. It also ensures that the correct student ID is sitting on the assigned seat by using face recognition. In addition, **ForeSyte** incorporates an Invigilator Activity Monitoring module to:

- Track invigilators' movements
- Detect prolonged idle periods
- Identify potential negligence

An admin dashboard provides real-time monitoring and automated report generation, enabling discipline committees to directly access timestamped violations instead of manually searching through hours of footage. By combining AI-powered surveillance with

intelligent analytics, **ForeSyte** enhances exam security and aims to reduce academic dishonesty over time by providing actionable insights. Furthermore, as students become aware that their activities are continuously monitored and every suspicious action is captured with timestamped reports, the likelihood of cheating is expected to decrease significantly over repeated examination sessions

Figure 1.1: Estimated reduction in cheating rates using ForeSyte AI-based surveillance system

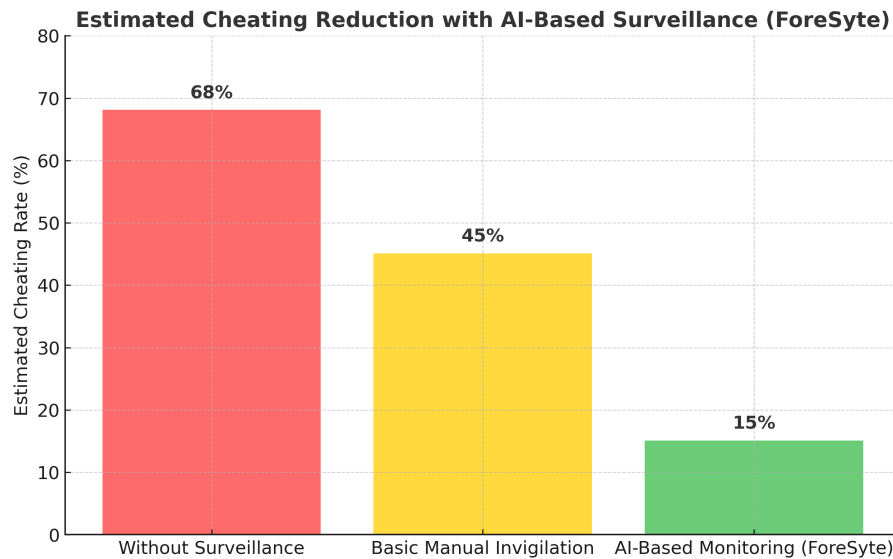
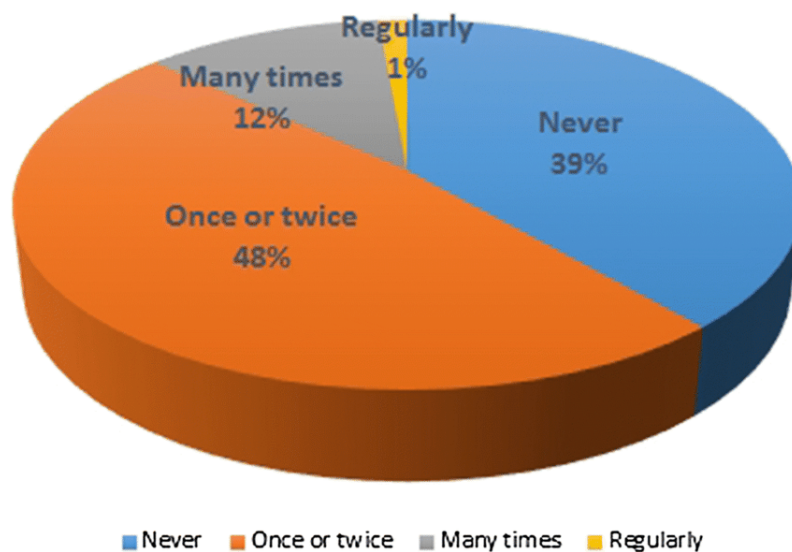


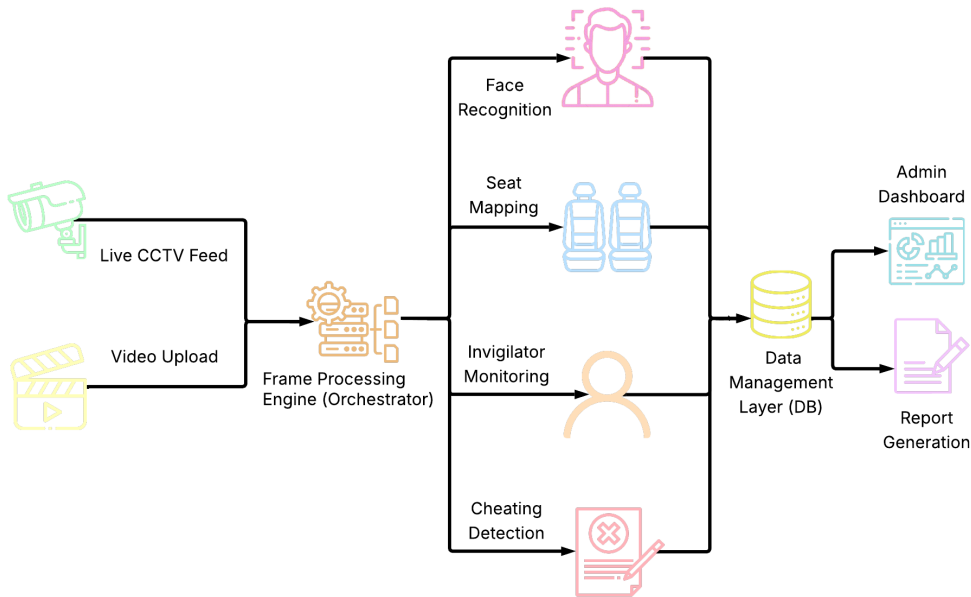
Figure 1.2: Frequency of students who reported cheating during written examinations [3]



1.3 Scope

The scope of the ForeSyte project includes developing an AI-powered exam surveillance system that enhances the integrity of examinations conducted in physical exam halls. Recent research highlights the importance of automated invigilation for maintaining academic integrity [3, 4].

Figure 1.3: Scope workflow of the ForeSyte system



The system will process CCTV/camera footage, both live streams and uploaded recordings, to automatically detect and log suspicious activities such as unauthorized object usage and irregular gestures. It will also incorporate seating arrangement verification, ensuring that each detected activity is accurately linked to the correct student, and that the correct student ID is sitting in the assigned seat using face recognition, similar to approaches in modern proctoring systems [1, 5].

The project further extends to monitoring invigilator activity within the exam hall, identifying negligence or prolonged absence from duty. Similar works on offline and intelligent monitoring systems support this approach [6, 7]. All detected events will be recorded with precise timestamps, seat numbers and room identifiers, enabling a structured and transparent record of violations.

An administrative dashboard will be developed to centralize monitoring and provide clear violation reports supported with visual evidence. This will reduce the need for institutions to manually review extensive CCTV/camera footage, offering instead a concise and accurate summary of flagged incidents.

The system will be designed for real-time performance using optimized frame sampling

and AI models capable of efficient video analysis. The project will be self-funded and developed entirely by our team. Related research also discusses the ethical implications of AI in exam supervision, highlighting fairness and privacy issues.

The boundaries of this project are confined to creating a robust and intelligent surveillance solution that automates detection, verification, and reporting within physical exam environments, thereby ensuring greater fairness, efficiency, and accountability in academic assessments.

1.4 Modules

The **ForeSyte** project is organized into several core modules, each responsible for specific functionalities required to ensure seamless, automated exam monitoring. The modules are divided into two main components: the *ForeSyte Web Portal* for stakeholders and the *Backend Service* that powers the AI-driven monitoring capabilities.

1.4.1 ForeSyte Web Portal

The **ForeSyte Web Portal** is a multi-user interface designed for administrators, discipline committee members, and university management. It provides centralized access to seating plans and violation reports, ensuring transparent and efficient management of examination activities.

1. **Dashboard Visualization:** Displays real-time updates of suspicious activities, flagged violations, and invigilator presence across multiple exam halls using data from connected CCTV/camera systems.
2. **Report Access:** Enables authorized users to view, manage, and download detailed violation reports containing timestamps, seat details and room information.
3. **User Roles and Permissions:** Implements secure role-based access, where administrators manage seating plans, camera configurations, and exam schedules, while discipline committee members focus on reviewing reports and initiating actions.
4. **Seating Plan Integration:** Links violations to student seating layouts and verifies correct student IDs using face recognition to ensure accurate identification of candidates involved in flagged incidents.

1.4.2 Backend Service

The **Backend Service** powers the core intelligence of the ForeSyte system, managing video analysis, behavioral detection, invigilator monitoring, and reporting. This module processes video streams from CCTV/camera systems and uploaded exam footage to ensure accurate real-time and recorded exam monitoring.

1. **Video Stream Processing:** Integrates with CCTV/camera systems and uploaded recordings, extracting frames for real-time detection of activities and events.
2. **Cheating Detection Engine:** Uses advanced computer vision techniques to identify suspicious behaviors such as unauthorized object usage, irregular gestures, communication, and frequent seat movements [1, 2].
3. **Face and Seat Verification:** Verifies student identities and ensures correct student ID is sitting on the assigned seat using face recognition, mapping detected activities to seating plans for accurate violation reporting.
4. **Invigilator Monitoring:** Tracks invigilator's prolonged absences from assigned zones, and identifies potential negligence or unauthorized phone usage[6].
5. **Report Generation:** Compiles timestamped violation logs, including room details, seat numbers, and integrates them directly into the ForeSyte Web Portal for automated PDF/Excel report exports.

1.5 Work Division

For each module and respective feature, responsibility is assigned to a team member.

Name	Responsibility
Inam Ullah Shaikh	• Frontend Development • Model Training • Documentation • Deployment • Backend Integration
Emaan Fatima	• Backend Development • Model Research • Model Training • Documentation • Deployment
Aden Sial	• Data Collection • Model Training • Model Research • Documentation • Deployment

Table 1.2: Work Division among Team Members

Chapter 2

Project Requirements [AS PER FYP1 MID REPORT]

This section outlines the necessary requirements for the successful completion of the project. Project requirements can be divided into two main categories: functional requirements, which describe the system's core operations, and non-functional requirements, which specify performance, security, and usability standards.

2.1 Use-case/Event Response Table/Storyboarding

This section describes how users will interact with the system through various scenarios, often referred to as use cases or event responses.

A use case represents a specific interaction between the user and the system, detailing the steps involved and the expected system responses. Each use case typically includes key components such as a unique identifier, a description of the interaction, the user or system actor involved, preconditions, the main flow of events, and postconditions.

Additionally, storyboarding can be used as a visual aid to depict the sequence of actions, illustrating how the user progresses through the system step by step. Storyboards are helpful for capturing the user experience and ensuring that the design aligns with the intended functionality.

Depending on the project, use cases, storyboarding or Event-response table can be chosen as appropriate methodologies to illustrate user interactions. Examples of all the approaches are provided in Appendix A.

2.2 Functional Requirements

This section describes the functional requirements of the system expressed in the natural language style. This section is typically organized by feature as a system feature name and specific functional requirements associated with this feature.

2.2.1 Module 1

Following are the requirements for module 1:

1. FR1
2. FR2

2.2.2 Module 2

Following are the requirements for module 2:

1. FR1
2. FR2

2.3 Non-Functional Requirements

This section specifies nonfunctional requirements. These quality requirements should be specific, quantitative, and verifiable. The following are some examples of documenting guidelines.

2.3.1 Usability

Usability requirements deal with ease of learning, ease of use, error avoidance and recovery, the efficiency of interactions, and accessibility. The usability requirements specified here will help the user interface designer create the optimum user experience. Example:

USE-1: The COS shall allow a user to retrieve the previous meal ordered with a single interaction.

2.3.2 Performance

State specific performance requirements for various system operations. If different functional requirements or features have different performance requirements, it's appropriate to specify those performance goals right with the corresponding functional requirements, rather than collecting them in this section. Example:

PER-1: 95% of webpages generated by the COS shall download completely within 4 seconds from the time the user requests the page over a 20 Mbps or faster Internet connection.

Chapter 3

System Overview [AS PER FYP1 MID REPORT]

Give a general description of the functionality, context, and design of your project. Provide any background information if necessary.

3.1 Architectural Design

Develop a modular program structure and explain the relationships between the modules to achieve the complete functionality of the system. This is a high-level overview of how the system's modules collaborate with each other to achieve the desired functionality.

Don't go into too much detail about the individual subsystems. The main purpose is to gain a general understanding of how and why the system was decomposed, and how the individual parts work together.

Provide a diagram showing the major subsystems and their connections.

- In the initial design stage, create a Box and Line Diagram for a simpler representation of the systems.
- After finalizing the architecture style/pattern diagram (MVC, Client-Server, Layered, Multi-tiered), create a detailed mapping of modules/components to each part of the architecture.

To view examples of box and line diagrams and architecture styles, see Appendix C.

3.2 Data Design

Explain how the information domain of your system is transformed into data structures. Describe how the major data or system entities are stored, processed, and organized.

List any databases or data storage items.

3.3 Domain Model

The domain model is a conceptual representation of the various entities, their attributes, and the relationships between them within the system. It serves as a bridge between the requirements and the design of the software, providing a clear understanding of the business domain and how it interacts with the system being developed.

A domain model typically includes key components such as:

- **Entities:** These are the primary objects within the domain that hold data and represent real-world concepts. Each entity is defined by its attributes and behaviors.
- **Attributes:** Characteristics or properties of the entities that define their state. For example, a "Customer" entity might have attributes such as `customerID`, `name`, `email`, and `address`.
- **Relationships:** These illustrate how different entities interact with each other. Relationships can be one-to-one, one-to-many, or many-to-many, depending on how entities are associated.
- **Associations:** Connections between entities that help clarify their interdependencies. For instance, a "Product" may be associated with a "Category," indicating the type of product it belongs to.

By constructing a domain model, developers can better understand the system's requirements, facilitate communication among stakeholders, and guide the subsequent design and implementation phases. It also aids in identifying potential issues early in the development process, ensuring a more robust architecture.

An example of a domain model for an online shopping system is provided in Appendix B.

3.4 Design Models [UPTO THE CURRENT ITERATION ONLY]

Create design models as are applicable to your system. Provide detailed descriptions with each of the models that you add. Also ensure visibility of all diagrams.

Design Models for Object Oriented Development Approach

The applicable models for the project using object oriented development approach may include:

- Activity Diagram
- Class Diagram
- Class-level Sequence Diagram
- [OPTIONAL] State Transition Diagram (for the projects which include event handling and backend processes)

Design Models for Procedural Approach

The applicable models for the project using procedural approach may include:

- Activity Diagram
- Data Flow Diagram (data flow diagram should be extended to 2-3 levels. It should clearly list all processes, their sources/sinks and data stores.)
- System-level Sequence Diagram
- [OPTIONAL] State Transition Diagram (for the projects which include event handling and backend processes)

Examples of above diagrams are given in Appendix D.

Chapter 4

Implementation and Testing

This chapter describes the current implementation status of the **ForeSyte** AI Exam Surveillance System and the testing performed up to this iteration. The focus of this phase is on the **Admin** and **Investigator** dashboards and their supporting backend services. Computer-vision based monitoring and the Student Portal are planned components and therefore remain out of scope for this iteration.

4.1 Current Progress

4.1.1 Front-End

The front-end of ForeSyte is implemented as a modern single-page application using React, TypeScript, Vite, and Tailwind CSS. The interface is split into role-specific areas so that administrators and investigators see different dashboards and workflows.

Admin Dashboard

The main admin dashboard presents a high-level overview of system activity. It displays:

- **Active Exams** – number of exams scheduled for the current or future dates.
- **Incidents Detected** – count of student activity records (currently using placeholder data; actual cheating detection is planned for future iterations).
- **Students Monitored** – number of students currently under surveillance based on seating plans and exam sessions.
- **Average Exam Duration** – average exam length computed from start and end times.

These statistics are fetched via the backend `/dashboard/stats` endpoint with a period parameter (today, week, or month). The dashboard also includes:

- An **Activity Overview** bar chart that compares the number of incidents and exams across time buckets using the `/dashboard/activity` endpoint.
- A **Recent Incidents** panel that lists the latest flagged student activities with severity tags using the `/dashboard/recent-incidents` endpoint.
- A **Quick Actions** area that links to live monitoring, reports, user management, and system settings.
- A **System Status** section that summarizes the health of the detection pipeline, camera network, analytics, and database.

The Admin Dashboard is therefore fully wired to the backend aggregation APIs. Loading states, error handling, and period switching are implemented in the front-end.

Investigator Portal

The Investigator Portal is implemented as a separate area in the React application, with its own header and sidebar components to clearly separate workflows from the admin role.

- The **Investigator Dashboard** displays summary cards for total violations, pending reviews, violations resolved today, and active exams. It also lists recent activities (e.g., phone usage, invigilator negligence, unauthorized movement) with severity and status badges and a period filter (today, week, month).
- The **Reports Page** shows a grid of generated reports with metadata such as report type (incident/exam), exam name, number of violations, generation date, and author. It provides filters for text search and report type, and a modal workflow for configuring a new report (type, export format, inclusion of video evidence links).

At this iteration, these investigator views use mock data to validate layout and interactions while backend report listing endpoints are being finalized. However, the UI contracts, filter behaviour, and state management are implemented and ready to be connected to live APIs.

4.1.2 Back-End

The backend is built with FastAPI and SQLAlchemy on top of PostgreSQL, with MongoDB used for flexible logging and high-volume events. Authentication uses JWT-based

tokens with role-based access control (RBAC) to differentiate between Admin, Investigator, Invigilator, and Student users.

Two backend modules are particularly important for this iteration:

- **Dashboard API** (/dashboard): aggregates data for the Admin and Investigator dashboards.
- **Reports API** (/reports): manages report generation and retrieval for incidents and exams.

Dashboard API

The /dashboard/stats endpoint computes:

- **Active Exams:** exams scheduled for the current or future dates.
- **Incidents Detected:** count of StudentActivity records in the selected period (note: actual cheating detection via computer vision is not yet implemented; this metric reflects activity records that would be populated by the detection pipeline in future iterations).
- **Students Monitored:** distinct count of students with seat assignments in active exams.
- **Average Exam Duration:** average duration (in minutes) based on start and end times of exams in the selected period.
- **Trends:** percentage changes for incidents, students, and exams when compared to the previous period.

The /dashboard/activity endpoint returns a sequence of time buckets, each containing the number of incidents and exams in that interval. For today, 4-hour slots are used; for week and month, daily buckets are used. The /dashboard/recent-incidents endpoint returns the most recent student activities, joined with student information to show names, types, severity levels, timestamps, and model confidence values.

All dashboard endpoints enforce RBAC policies and only allow access to users with admin or investigator roles.

Reports API

The /reports router provides the API structure for managing report records. **Note:** The actual file generation (PDF, CSV, Excel) is not yet implemented in this iteration;

the endpoints currently create database records and return metadata, but do not produce downloadable report files.

- Standard CRUD endpoints (POST /, GET /, GET /{id}, PUT /{id}, DELETE /{id}) operate on the Report model and validate related Violation and Investigator records. These endpoints are functional for managing report metadata.
- POST /reports/incidents accepts a list of incident IDs, desired export format, and whether to include video links. It validates and resolves the incident IDs into StudentActivity records, ensures there is an associated Violation, creates a new Report entry in the database, and returns a JSON response containing the report ID, file path, format, and status. **The actual report file generation (PDF/CSV/Excel) is planned for a future iteration.**
- POST /reports/exams/{exam_id} is designed to generate an exam-level report by aggregating all StudentActivity entries for the given exam and producing a Report record. **Currently, this endpoint creates the database record but does not generate the actual report file.**

These APIs establish the data model and workflow structure for report management, but the actual document generation logic will be implemented in subsequent iterations.

4.1.3 Next Steps

The completed work in this iteration establishes the core dashboard infrastructure and API structure for reports. Upcoming work will:

- Connect the Investigator Portal to live dashboard and report APIs instead of mock data.
- Implement actual report file generation (PDF, CSV, Excel) for incident and exam reports.
- Integrate the video processing pipeline (camera ingestion, AI models, and real-time cheating detection) into the monitoring endpoints.
- Finalize and expose the Student Portal, including student-facing views of violation histories and exam schedules.

4.2 Algorithm Design

This section summarizes the main algorithms and data flows used to support the implemented features in ForeSyte.

4.2.1 Admin Dashboard Aggregation

The Admin Dashboard relies on server-side aggregation logic to compute statistics for different time windows. The high-level algorithm for computing dashboard statistics is:

1. **Determine Time Window:** Given $\text{period} \in \{\text{today}, \text{week}, \text{month}\}$, compute the current window (`start_date`) and the immediately preceding window (`prev_start`, `prev_end`) for trend comparison.
2. **Count Active Exams:** Query Exam where `exam_date` is greater than or equal to the current date. Perform the same query on the previous window to obtain `prev_active_exams`.
3. **Count Incidents and Students:**
 - Incidents: count `StudentActivity` records with timestamps within the current window.
 - Previous incidents: count `StudentActivity` records within the previous window.
 - Students monitored: count distinct `Seat.student_id` in rooms and exams that fall inside the current and previous windows.
4. **Compute Average Exam Duration:** For all exams in the current window with non-null start and end times, compute the duration in minutes and average the result.
5. **Compute Trends:** For each metric (incidents, students, exams), compute percentage change between current and previous counts, handling the edge case when the previous value is zero.
6. **Return Structured Response:** Package all calculated values into a `DashboardStats` object and return it to the client.

On the client side, the React dashboard:

- Calls `/dashboard/stats` to populate the metric cards.

- Calls `/dashboard/activity` to render a bar chart comparing incidents and exams per time bucket.
- Calls `/dashboard/recent-incidents` to display the latest suspicious activities with relative time labels.

4.2.2 Report Generation Workflow (Planned Design)

The report generation workflow is designed to be configurable and format-agnostic. **Note:** The following describes the planned design; actual file generation (PDF, CSV, Excel) is not yet implemented in this iteration.

Incident Reports (Planned)

1. The investigator selects one or more incident IDs and chooses an export format (e.g., PDF, CSV, JSON), with options such as including video evidence links.
2. The front-end constructs an `IncidentReportRequest` and sends it to `POST /reports/incidents`.
3. The backend validates permissions, resolves the incident IDs into `StudentActivity` records, ensures there is an associated `Violation`, and generates a unique report filename.
4. A new `Report` record is inserted in the database, and the API returns the report ID, file path, format, and status to the client. **In the current iteration, the actual report file is not generated; only the database record is created.**

Exam Reports (Planned)

1. The investigator selects an exam and desired output format from the Reports Page.
2. The front-end sends an `ExamReportRequest` to `POST /reports/exams/{exam_id}`.
3. The backend verifies the exam, aggregates its related `StudentActivity` records, ensures a linked `Violation` exists, and creates a `Report` entry in the database.
4. The API responds with the report metadata. **The actual report file generation (PDF/CSV/Excel) will be implemented in a future iteration.**

4.2.3 Investigator Violation Review Flow

The Investigator Dashboard is designed to give investigators a structured way to triage violations:

1. Load dashboard summary counters (total violations, pending reviews, resolved today, active exams).
2. Display a list of recent activities with type, severity, and status tags.
3. Allow investigators to toggle the period filter to focus on today, week, or month.
4. Provide a *Review* action for each activity which, in the next iteration, will navigate to a detailed case view and allow updating violation status and adding investigation notes.

4.3 External APIs and Libraries

At this stage, ForeSyte does not depend heavily on third-party AI services, but it makes extensive use of framework libraries and SDKs:

- **FastAPI** and **Starlette** for HTTP routing, dependency injection, and request handling.
- **SQLAlchemy** and **psycopg2** for ORM and PostgreSQL connectivity.
- **PyMongo** for MongoDB integration in the logging and evidence pipeline.
- **OpenCV** and image processing libraries, which are already installed and used in internal test scripts to handle frames for future AI detectors.
- **React Router**, **Tailwind CSS**, and **lucide-react** on the front-end for routing, styling, and icons.

4.4 Testing Details

Testing for this iteration concentrates on validating backend APIs, role-based access control, and the correctness of aggregated statistics, along with front-end behaviour under typical and edge-case conditions.

4.4.1 Unit Testing

Reports API Tests

Table [4.1](#) summarizes representative unit tests for the Reports API.

Table 4.1: Unit Test Cases for Reports API

Test ID	Object	Preconditions	Steps	Test Data	Expected Result	Postconditions	Actual Result	Pass/Fail
TC201	Verify admin can create a report	Valid admin JWT; existing Violation and Investigator records	Call POST /reports with valid payload	report_type: "incident"				
violation_id: valid UUID generated_by: valid UUID	Return 201 Created and a Report object with non-null report_id	Database contains a new Report row	As Expected	Pass				
TC202	Enforce RBAC on report creation	Authenticated investor JWT	Call POST /reports with valid payload	Same as TC201	Return 403 Forbidden with "Only admins can create reports"	No new Report row is created	As Expected	Pass

Dashboard API Tests

Table [4.2](#) lists key unit tests for the dashboard statistics and recent incidents endpoints.

Table 4.2: Unit Test Cases for Dashboard APIs

Test ID	Objective	Preconditions	Steps	Test Data	Expected Result	Postconditions	Actual Result	Pass/Fail
TC301	Retrieve “to-day” dashboard stats	Admin JWT; at least one exam scheduled today	Call GET /dashboard/stats	period: "to-day"	Returns a valid JSON object; all counts are non-negative and consistent with seeded data	Values computed from sample database state	As Expected	Pass
TC302	Verify period filter behaviour	Seeded data across multiple days	Call GET /dashboard/stats with different periods	period: "to-day", "week", "month"	Returns different counts reflecting the date range; no errors occur	Internal trend computation behaves correctly	As Expected	Pass
TC303	Enforce RBAC on stats	Student JWT	Call GET /dashboard/stats	period: "to-day"	Returns 403 Forbidden and “Access denied”	No sensitive stats are exposed	As Expected	Pass
TC304	Return recent incidents list	Student JWT	Call GET /dashboard/recent-incidents	limit: 5 27	Returns up to 5 incidents ordered by times-tamp;	Activities are ordered from most recent to oldest	As Expected	Pass

Front-End Behaviour Tests

The front-end was validated using component-level tests and manual exploratory runs:

- **Admin Dashboard Rendering:** Ensured that the metric cards, activity chart, and recent incidents list render correctly when provided with sample API responses, and that loading and empty states are displayed when data is unavailable.
- **Investigator Dashboard Layout:** Checked that period filters toggle correctly, summary counters are displayed, and recent activity items show appropriate severity and status chips.
- **Reports Page Filtering:** Verified that text search and report-type filters reduce the visible set of reports appropriately and that the “Generate New Report” modal captures the selected configuration.

In addition, a dedicated test script was developed for testing the phone stream ingestion and processing pipeline without enabling AI-based detection. This script validates connection to phone camera streams and basic frame capture, ensuring that the underlying streaming components are stable and ready for integration with the full monitoring pipeline in subsequent iterations. **Note:** Actual cheating detection via computer vision models is not yet implemented; the test script focuses on connectivity and frame handling only.

User Management API Tests

Table 4.3 summarizes unit tests for the User Management API, which allows administrators to create, view, update, and delete users across all roles (Admin, Investigator, Invigilator, Student).

Table 4.3: Unit Test Cases for User Management API

Test ID	Object	Precondition	Setup	Test Data	Expected Result	Postcondition	Actual Result	Pass/Fail
TC401	Verify admin can create a new student user	Valid admin JWT, with database access-sible	Call POST /users	name: "John Doe"				
	email: "john@example.com" user_type: "student" roll_number: "22I-1234" password: "secure123!"							
		Return 201 Created with User object containing non-null id, email, roll_number and roll_number	New Student created with record exists in database with hashed password, email, roll_number	As Expected	Pass			
TC402	Verify admin can create an inves-ti-gator user	Valid admin JWT, with database access-sible	Call POST /users	name: "Jane Smith"				

Seating Plan Upload Tests

Table 4.4 lists test cases for the seating plan upload functionality, which processes PDF files containing exam schedules and extracts room, student, and seat assignment information.

Table 4.4: Unit Test Cases for Seating Plan Upload

Test ID	Object	Pre-conditions	Steps	Test Data	Expected Result	Post-condition	Audit Re-sult	Pass/Fail
TC501	Upload valid seating plan PDF file; ad-plan.pdf	Valid Call file: POST seat-ing ad-plan PDF JWT; file with database room ac-ces-sible	Room As Ex-pected Seat records cre-ated/updated in room database num-ber, exam date, time slot, and list of stu-dents with seat num-bers	Return 200 OK with ex-tracted data in-clud-ing room num-ber, exam date, time slot, and list of stu-dents with seat num-bers	Pass			
TC502	Upload valid seating plan PDF file; ad-plan.pdf	Valid Call file: POST seat-ing ad-plan PDF JWT; file with database room ac-ces-sible	Room As Ex-pected Seat records cre-ated/updated in room database num-ber, exam date, time slot, and list of stu-dents with seat num-bers	Return 200 OK with ex-tracted data in-clud-ing room num-ber, exam date, time slot, and list of stu-dents with seat num-bers	Pass			

Monitoring and Camera Integration Tests

Table 4.5 presents test cases for the monitoring endpoints and camera integration functionality, which enable administrators and investigators to view live camera feeds and manage monitoring sessions.

Table 4.5: Unit Test Cases for Monitoring and Camera Integration

Test ID	Object	Preconditions	Steps	Test Data	Expected Result	Postconditions	Actual Result	Pass/Fail
TC601	Retrieve monitoring feeds for all cameras	Admin JWT; rooms with cameras exist in database	Call GET /monitoring/	No parameters	Returns 200 OK with list of camera feeds including camera ID, room name, status, stream URL, and students monitored count	Response contains valid feed data for all rooms	As Expected	Pass
TC602	Filter monitoring feeds by exam ID	Admin JWT; multiple exams with rooms	Call GET /monitoring/?exam_id={exam_id}	exam_id: valid ID	Returns only camera feeds for rooms associated with the specified exam_id	Response contains filtered feeds by exam	As Expected	Pass

Bibliography

- [1] Saravanan Arumugam. Deep learning-based smart invigilation system for enhanced exam integrity. *Proceedings of Engineering and Technology Innovation*, 29:99–115, 2025.
- [2] A. Bhatti and T. Aslam. Smart exam monitoring system using computer vision. *Journal of Emerging Technologies*, 2023.
- [3] A. Buccioli, S. Cicognani, and N. Montinari. Cheating in university exams: the relevance of social factors. *ResearchGate*, 2020.
- [4] M. Khalid and H. Shah. Impact of surveillance systems on academic integrity in higher education. *Journal of Educational Research*, 2021.
- [5] Mohammad M. Masud, Khadim Hayawi, Sujith Samuel Mathew, Temesgen Mikael Abraha, and Mai ElBarachi. Smart online exam proctoring assist for cheating detection. *ResearchGate*, 2025.
- [6] Reuben Moyo, Stanley Ndebvu, Michael Zimba, and Jimmy Mbelwa. A video-based detector for suspicious activity in examination with openpose. *Unknown Journal*, 2023.
- [7] –. Intelligent offline exam monitoring system (iems) for identifying suspicious behavior. *International Journal of Computer Applications*, 2025.

Appendix A

Appendices

A.1 Appendix A

A.1.1 Use Case Diagram example (Online Shopping System)

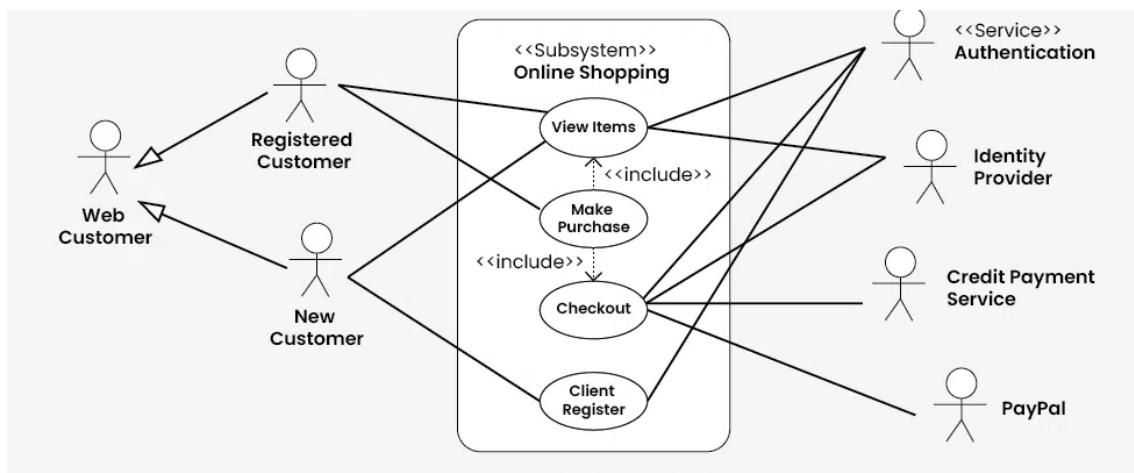


Figure A.1: Use Case Diagram for the Online Shopping System

A.1.2 Detail Use Case Example

ID	#1
Name	Overview elements
Short Description	All elements are shown in a list, where the user can set different filters.
Goal	Displaying elements in a changeable view.
Preconditions	The user got access to the system and is logged in. The user got the right to see schedules.
Success End Condition	The correct elements (filter and sorting) are displayed.
Fall End Condition	Elements not matching the criteria are displayed.
Stakeholder	Customer manager
Trigger	Login in as customer manager (because overview is on the starting screen for this role).
Normal Flow	1. The list of all elements matching default criteria are shown. 2. The user changes the filter criteria. 3. The user presses the Filter button. 4. The list shows all matching elements. Optional: 5. The user presses the Clear button. 6. The list of all elements matching default criteria are shown.
Alternative Flows	With click on the column headers, the list sorting can be changed. Different sorting for the different columns is described in the table below.
Includes	Login
Frequency of Use	About 50 times per day
Constraints and Special Requirements	None
Assumptions	None
Notes and Issues	None

Figure A.2: Detail Use Case Example

A.1.3 Event-Response Table for a Highway Intersection

Event	System State	Response
Road sensor detects vehicle entering left-turn lane.	Left-turn signal is red. Cross-traffic signal is green.	Start green-to-amber countdown timer for cross-traffic signal.
Green-to-amber countdown timer reaches zero.	Cross-traffic signal is green.	1. Turn cross-traffic signal amber. 2. Start amber-to-red countdown timer.
Amber-to-red countdown timer reaches zero.	Cross-traffic signal is amber.	1. Turn cross-traffic signal red. 2. Wait 1 second. 3. Turn left-turn signal green. 4. Start left-turn-signal countdown timer.
Pedestrian presses a specific walk-request button.	Pedestrian sign is solid Don't Walk. Walk-request countdown timer is not activated.	Start walk-request countdown timer.
Pedestrian presses walk-request button.	Pedestrian sign is solid Don't Walk. Walk-request countdown timer is activated.	Do nothing.
Walk-request countdown timer reaches zero plus the amber display time.	Pedestrian sign is solid Don't Walk.	Change all green traffic signals to amber.
Walk-request countdown timer reaches zero.	Pedestrian sign is solid Don't Walk.	1. Change all amber traffic signals to red. 2. Wait 1 second. 3. Set pedestrian sign to Walk. 4. Start don't-walk countdown timer.

Figure A.3: Example of Event Response Table

A.1.4 Story Board Example For Android App

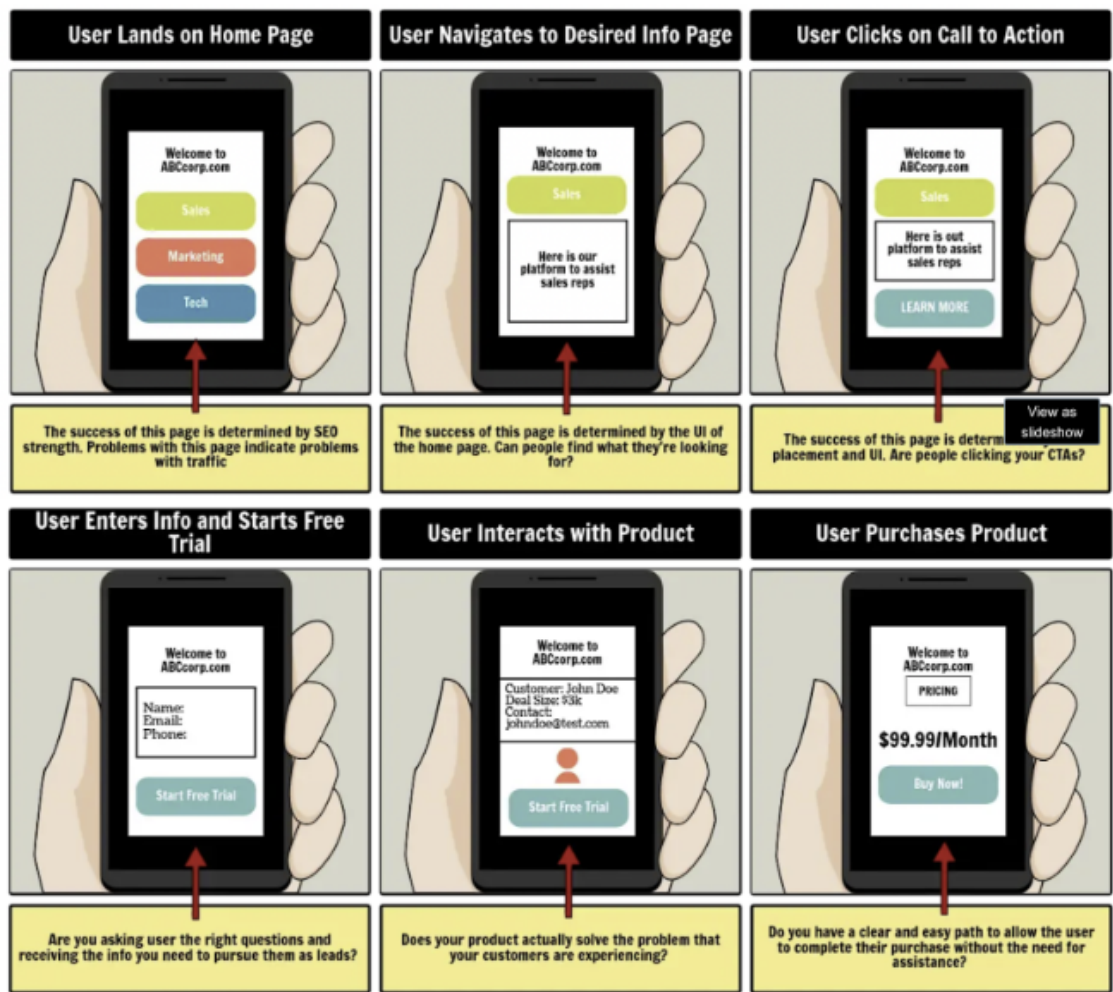


Figure A.4: Example of Story Board

A.3 Appendix C

A.3.1 Box And Line Example For Online Shopping

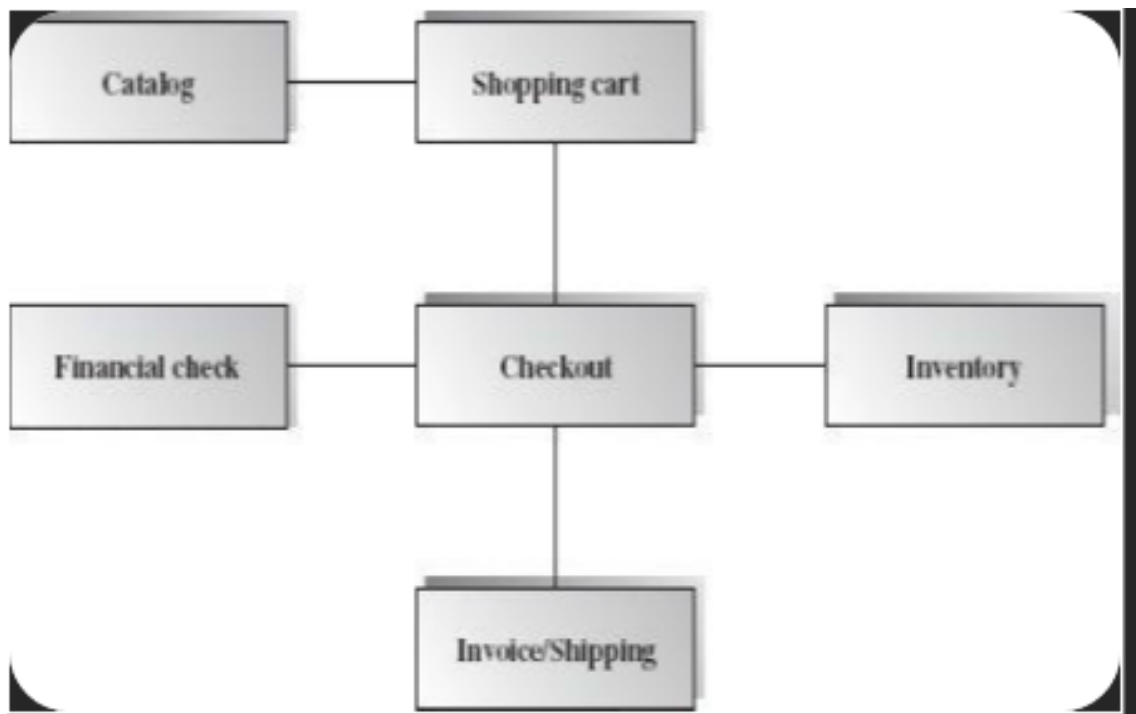


Figure A.6: Box and Line Diagram For Online Shopping Application

A.3.2 Architecture Pattern Example For Online Shopping

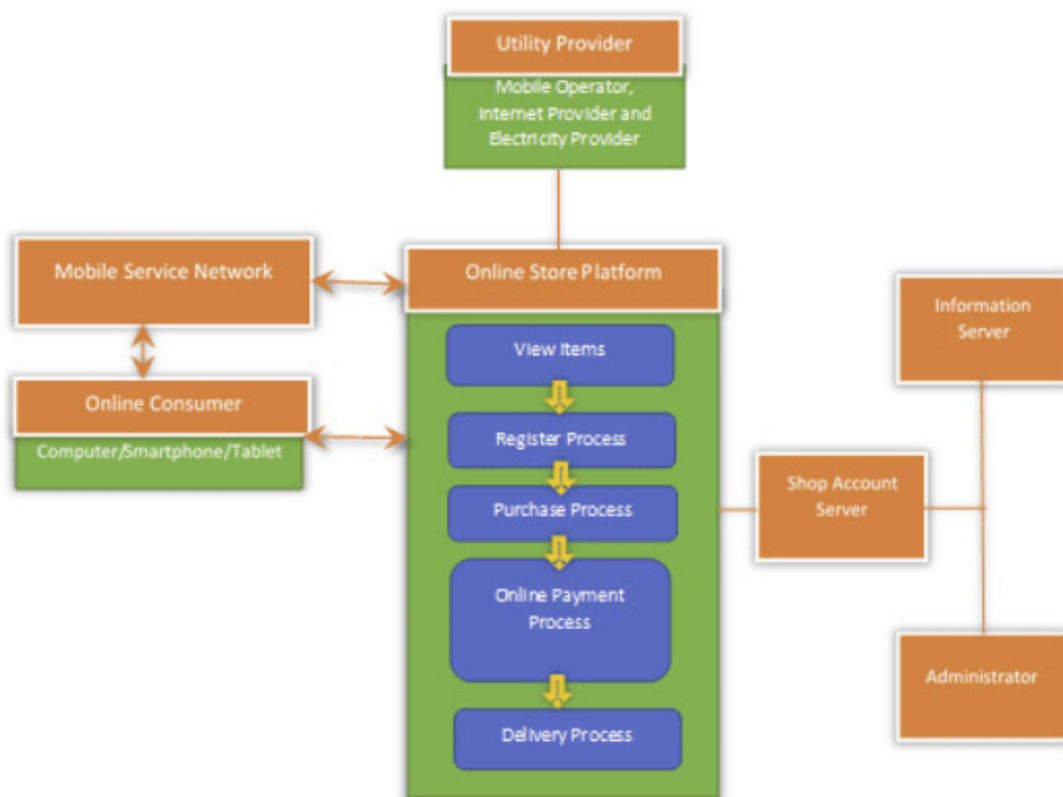


Figure A.7: Architecture Pattern For Online Shopping Application

A.4 Appendix D

A.4.1 Activity Diagram

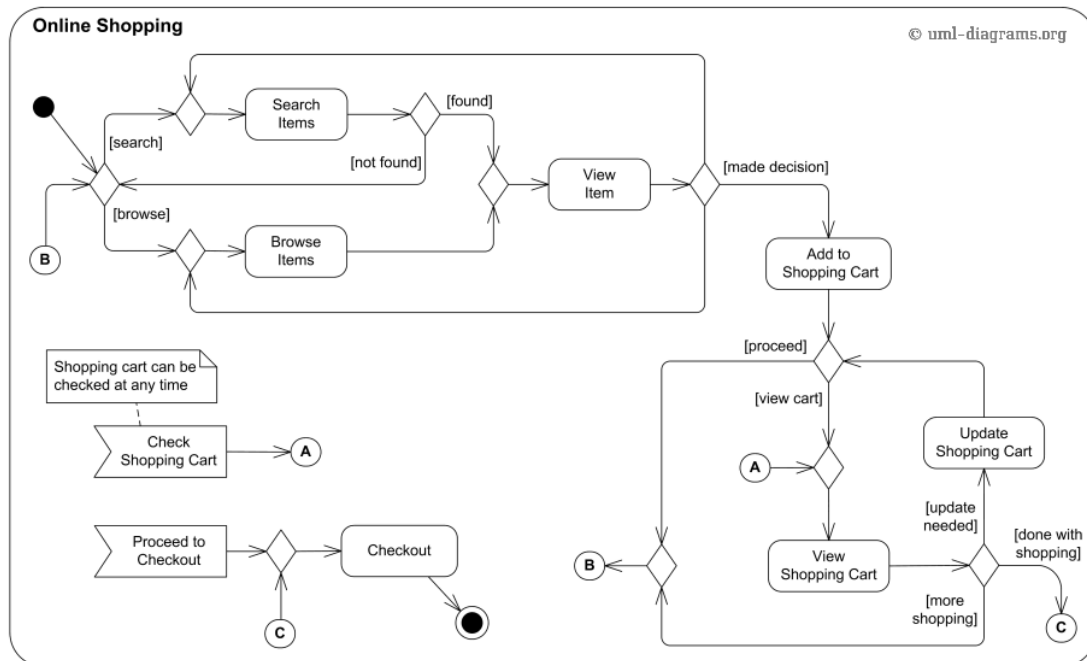


Figure A.8: Activity Diagram For Online Shopping Application

A.4.2 Class Diagram

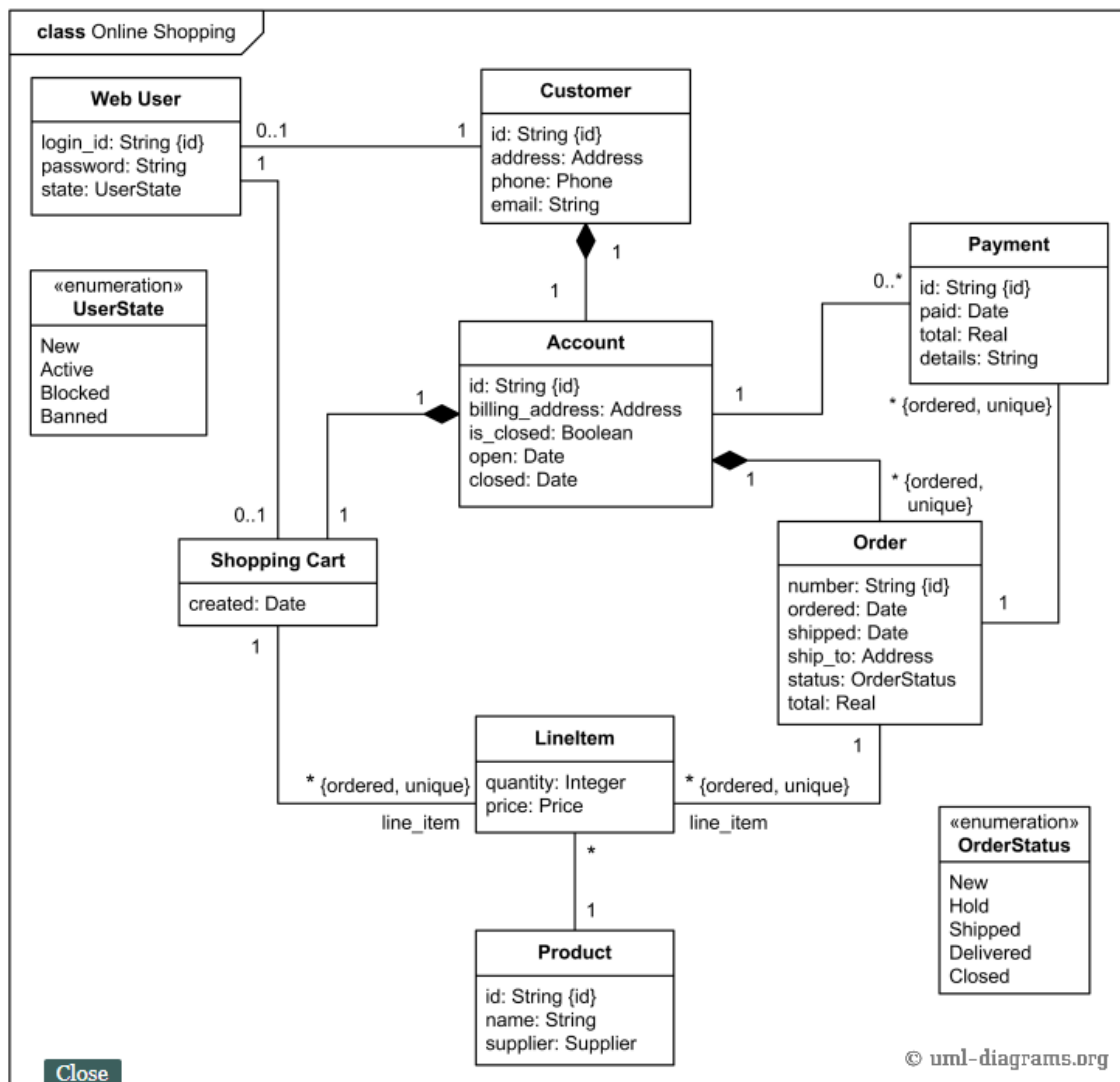


Figure A.9: Class Diagram For Online Shopping Application

A.4.3 Sequence Diagram

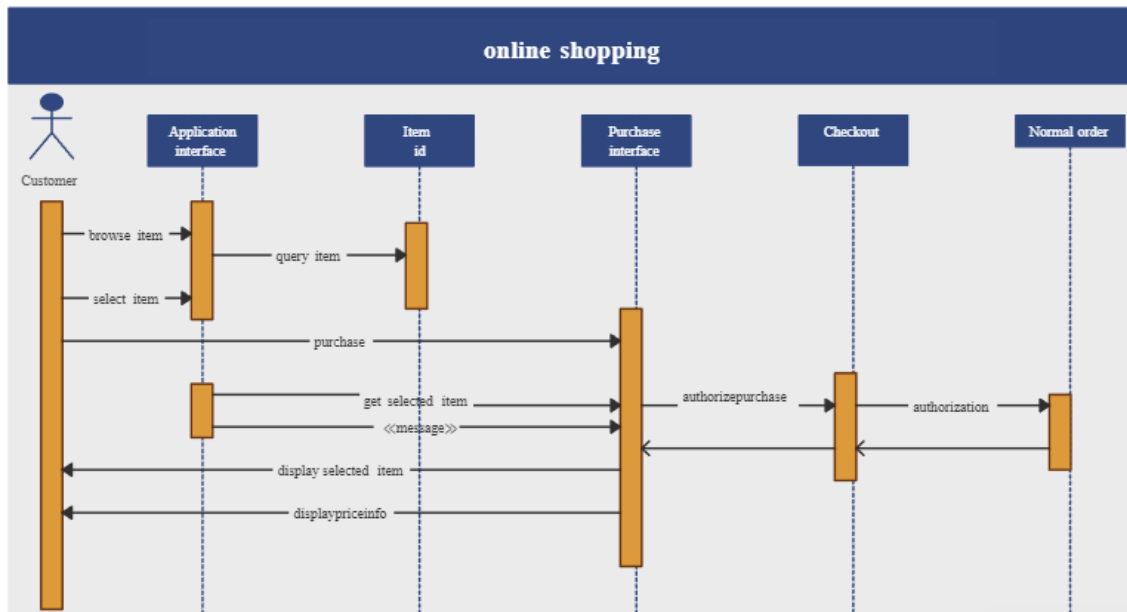


Figure A.10: Sequence Diagram For Online Shopping Application

A.4.4 State Transition Diagram

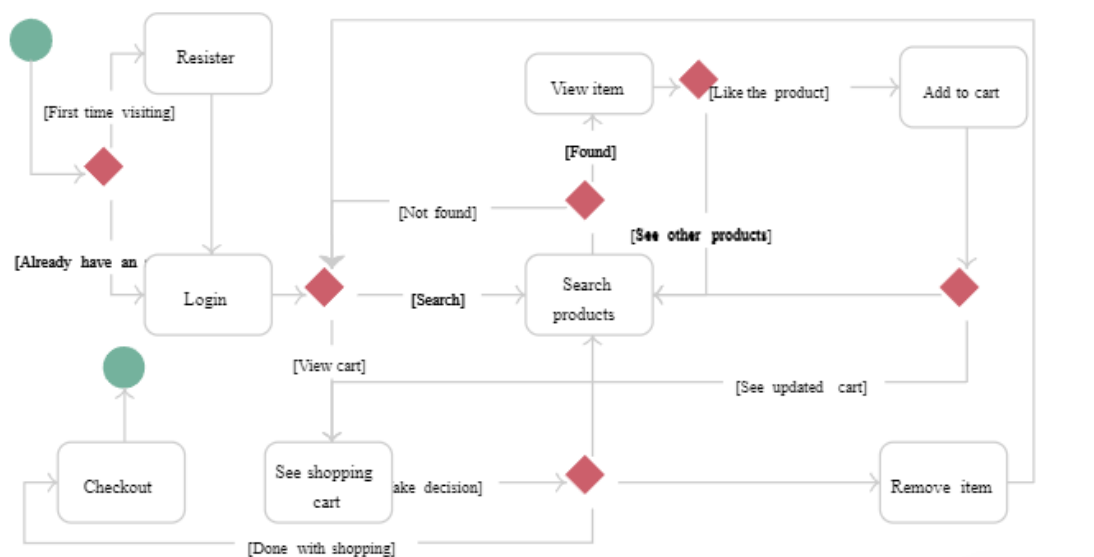


Figure A.11: State Transition Diagram For Online Shopping Application

A.4.5 Data Flow Diagram

Data Flow Diagram

Data flow diagram symbols, symbol names, and examples

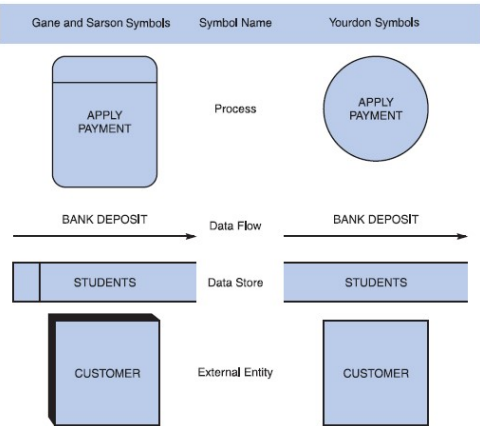


Figure B-5 Data flow diagram symbols, symbol names, and examples of the Gane and Sarson and Yourdon symbol sets.

Guidelines for Drawing DFDs

Step 1: Draw a Context Diagram: The first step in constructing a set of DFDs is to draw a context diagram. A **context diagram** is a top-level view of an information system that shows the system's boundaries and scope. Data stores are not shown in the context diagram because they are contained within the system and remain hidden until more detailed diagrams are created.

Example

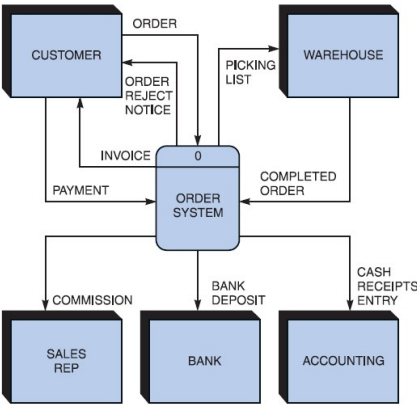


Figure B-6 Context diagram DFD for an order system.

Step 2: Draw a Diagram 0 DFD: To show the detail inside the black box, you create DFD diagram 0. **Diagram 0** zooms in on the system and shows major internal processes, data flows, and data stores. Diagram 0 also repeats the entities and data flows that appear in the context diagram. When you expand the context diagram into DFD diagram 0, you must retain all the connections that flow into and out of process 0.

Example

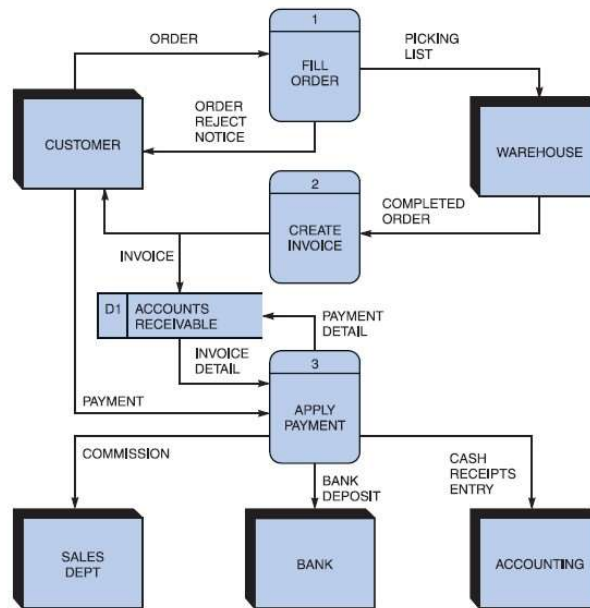


Figure B-7 Diagram 0 DFD for the order system.

Step 3: Draw the Lower-Level Diagrams:

To create lower-level diagrams, you must use leveling and balancing techniques. **Leveling** is the process of drawing a series of increasingly detailed diagrams, until all functional primitives are identified.

Leveling Example

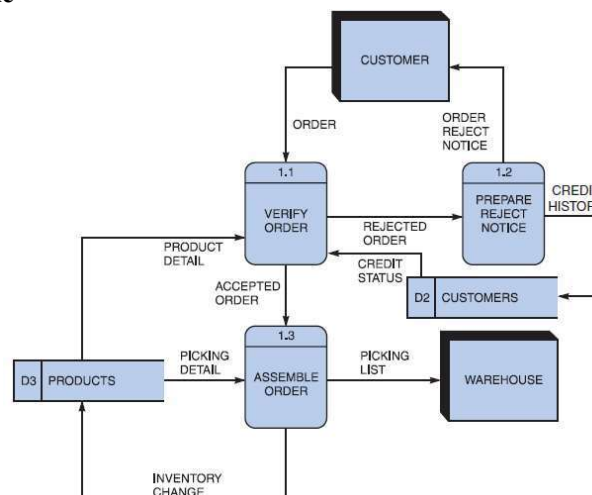


Figure B-8 Diagram 1 DFD shows details of the FILL ORDER process in the order system.